MRI Foreground/Background Detection

Amir R. Sadri
ars329@case.edu
December 13, 2025

Abstract

This note outlines how we derive foreground and background masks for MRI volumes in RadQy. Each section documents one detection method, its assumptions, and the clean-up steps needed so FG/BG pixels are reproducible across datasets. sarvar

Contents

1 Introduction

This document lists the foreground/background (FG/BG) segmentation methods in `backend/seg/mri`. Each section mirrors a Python module and notes its inputs, thresholds, morphology, and fallbacks so downstream IQMs get consistent masks.

Unless noted, we normalize each MRI slice to uint8 with dynamic range $[0, 255]$ before thresholding. Inputs outside this range are clipped, and all downstream FG/BG masks are therefore binary and non-negative. Any negative-valued inputs are treated as invalid and skipped from analysis.

2 Image Processing Segmenters

Classical pipelines based on intensity statistics and morphology.

2.1 OtsuHull (`OtsuHull.py`)

Legacy variant mirroring the original hybrid Otsu pipeline with more defensive fallbacks [1]. This subsection is listed first because it matches the earliest implementation shipped with RadQy.

Algorithm 1: OtsuHull foreground/background estimation

1. Input slice I; cast to uint16.
2. Histogram-equalize I to get H.
3. Otsu on I and H to obtain masks m_1, m_2 and FG fractions w_1, w_2.
4. Hybrid recombination: $S = w_1 \cdot I + w_2 \cdot H$ (normalize to image range).
5. Otsu on S to get final mask M.
6. Cleanup: convex hull of M (fallback: hole filling if hull fails).
7. Outputs: $fg = M$, $bg = 1 - M$.

Notes on the weights and masks: m_1 is the Otsu mask on the raw slice I, m_2 is the Otsu mask on the hist-equalized slice H. The weights $w_1 = |m_1|/|I|$ and $w_2 = |m_2|/|I|$ measure how much of each slice is classified as foreground; they scale the raw and equalized images when forming $S = w_1 \cdot I + w_2 \cdot H$ so that the cleaner mask (smaller foreground fraction) influences the recombined image more.

2.2 Adaptive Border Segmentation (`AdaptiveBorderSeg.py`)

Single-slice segmenter that estimates background from the image border and keeps the largest non-background component.

Algorithm 2: AdaptiveBorder foreground/background estimation

1. Normalize input slice to uint8 via 1st/99th percentile scaling.
2. Extract border ring (default 12 px each side); compute μ, σ on non-zero border pixels (fallback: full image if border empty).
3. Mark background where $I < \mu - 1\sigma$.
4. Morphology: binary closing then opening with 5×5 kernel; fill holes.
5. Foreground = inverse of background; keep largest connected component.
6. Safety: if FG area $< 5\%$, set FG to all-ones (entire slice).
7. Outputs: (fg, bg) with $bg = 1 - fg$.

2.3 Hybrid Otsu + Convex Hull (`HybridOtsuHullSeg.py`)

Modern hybrid approach combining Otsu on raw and histogram-equalized images with convex hull cleanup.

Algorithm 3: Hybrid Otsu + Convex Hull

1. Robust-scale input slice to $[0, 1]$ via 1st/99th percentiles; cast to uint8 for thresholding.
2. Compute Otsu masks on raw (x) and hist-equalized (h) images to get FG fractions w_1, w_2.
3. Recombine: $S = w_1 \cdot x + w_2 \cdot h$.
4. Otsu on S to obtain mask M.
5. Cleanup: apply convex hull to M (fallback: hole filling if hull unavailable).
6. Outputs: (fg, bg) with $fg = M$, $bg = 1 - fg$.

3 Deep Learning Segmenters

Learned models for FG/BG inference.

3.1 U-Net

Learned U-Net inference using the default checkpoint at `backend/seg/mri/models/unet.pt`. The samples generated by `tests/sample_random_seg_masks.py` are saved under `docs/tex/Figure/MRI/seg/unet`.

`num=1, scantype=MRI, bodypart=Pelvis, im=Figure/MRI/seg/unet/Pelvis_T2_AX_23_im.png, fg=Figure/MRI bg=Figure/MRI/seg/unet/Pelvis_T2_AX_23_bg.png`

hi fouad

ss

References

[1] MRQy—An open-source tool for quality control of MR imaging data. *Medical physics*, 2020.